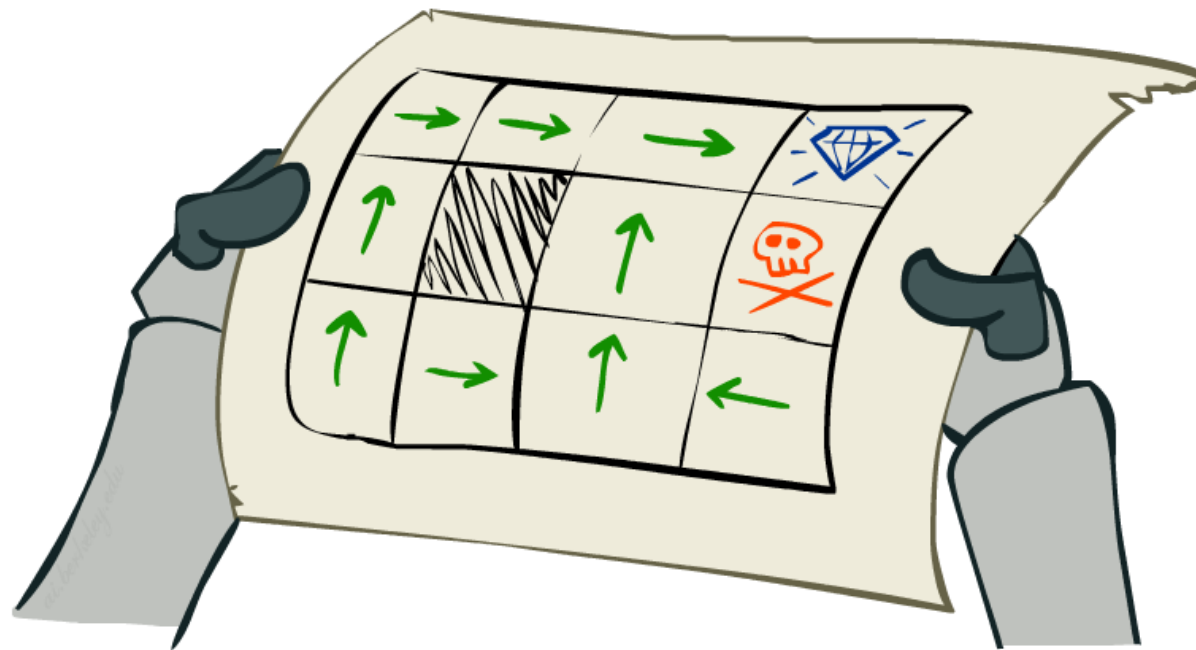
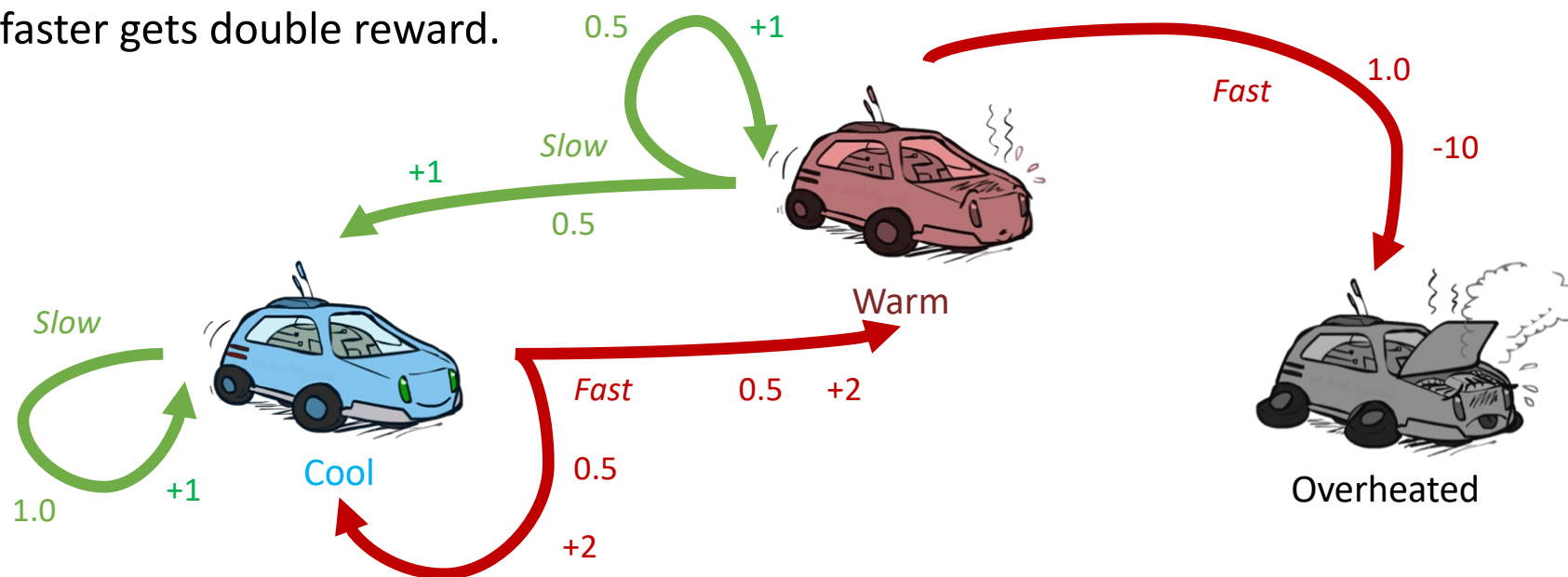


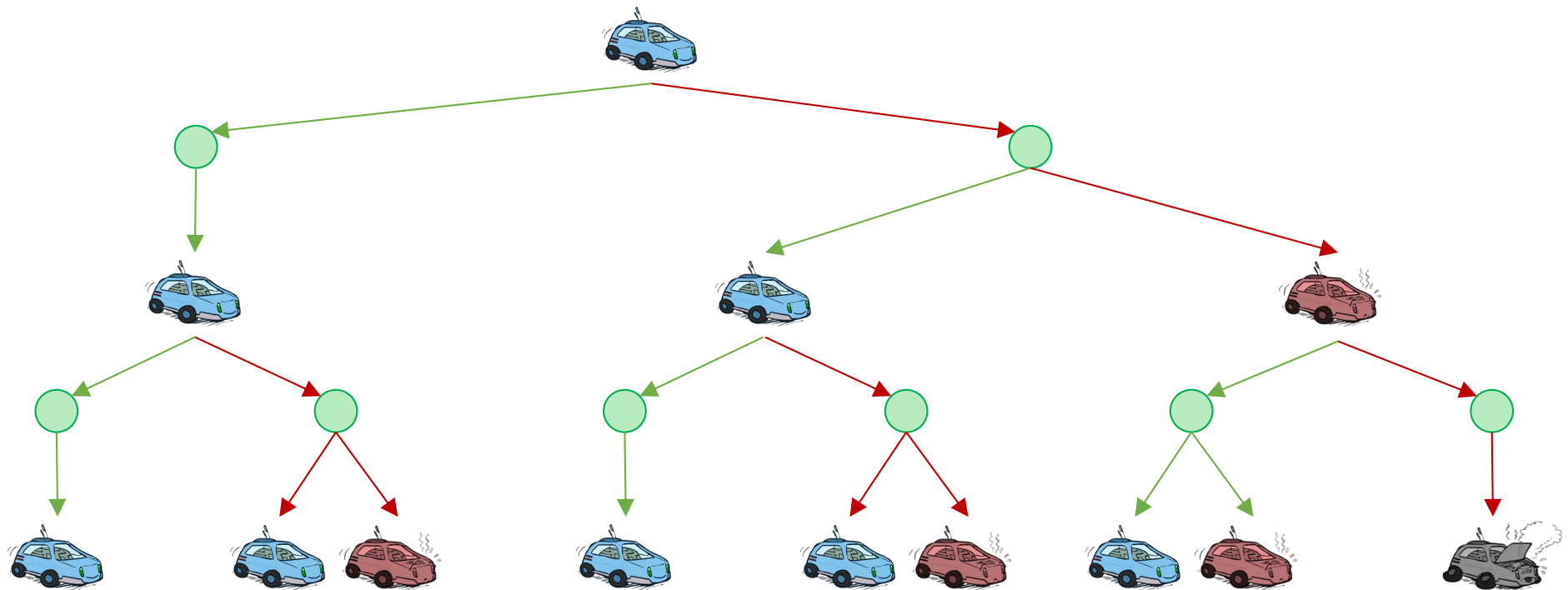
Solving MDPs



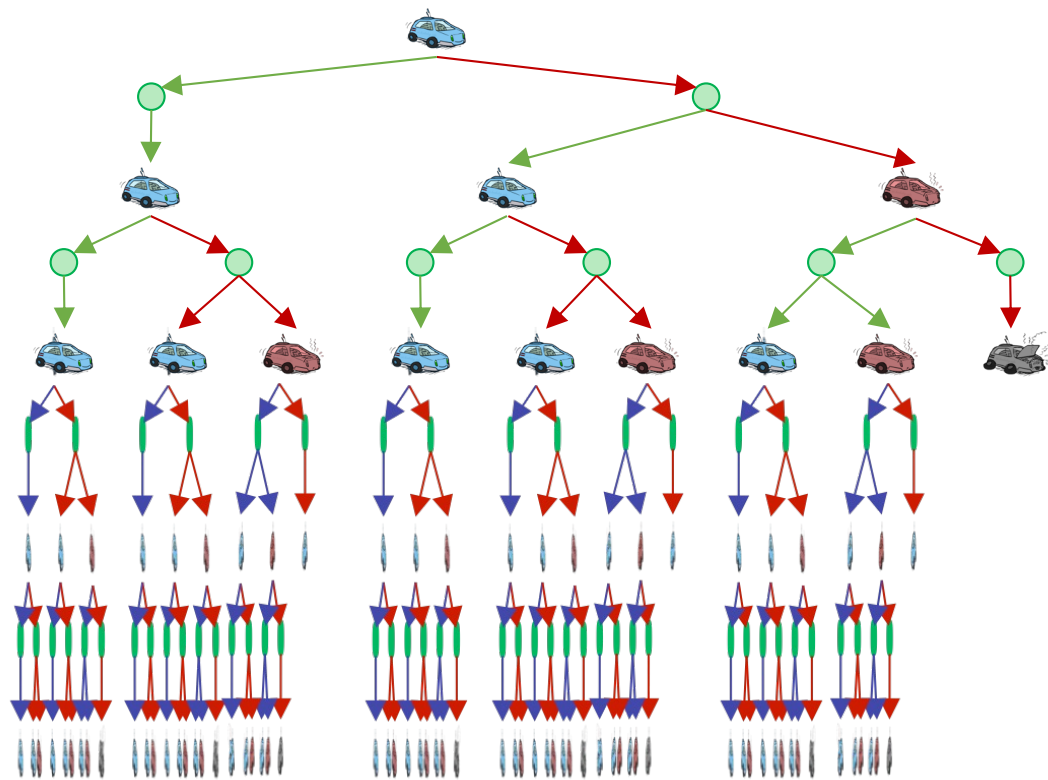
Recall: Racing MDP

- A robot car wants to travel far, quickly.
- Three states: *Cool*, *Warm*, *Overheated*.
- Two actions: *Slow*, *Fast*.
- Going faster gets double reward.





Racing Search Tree

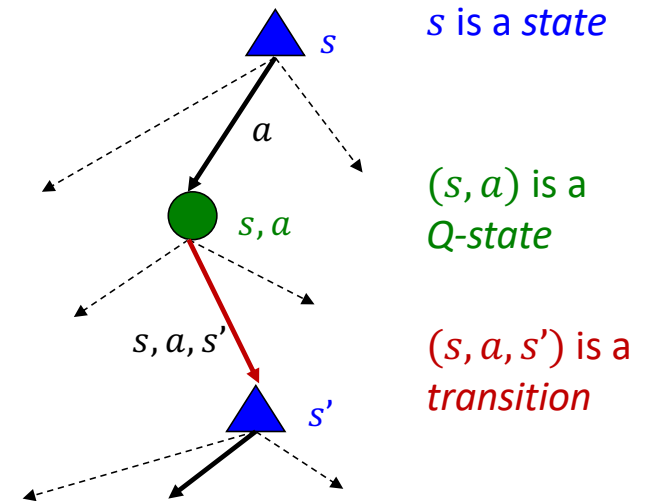


Racing Search Tree

- Why Not Just Use Expectimax?
 - Redundant Computation: States are revisited multiple times across branches;
 - Infinite Tree Depth: The search tree can grow indefinitely.
- Smarter Strategy
 - Memoization: Only compute each state's value once, and reuse it;
 - Depth-Limited Updates: Perform updates with increasing depth, stopping when changes become negligible;
 - Discounting Helps: With $\gamma < 1$, deeper parts of the tree contribute less to total value.

Optimal Quantities

- The value (utility) of a **state** s :
 - $V^*(s)$ = expected utility starting in s and acting optimally.
- The value (utility) of a **Q-state** (s, a) :
 - $Q^*(s, a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally.
- The optimal **policy**:
 - $\pi^*(s)$ = optimal action from state s .

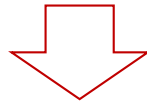


Bellman Equation

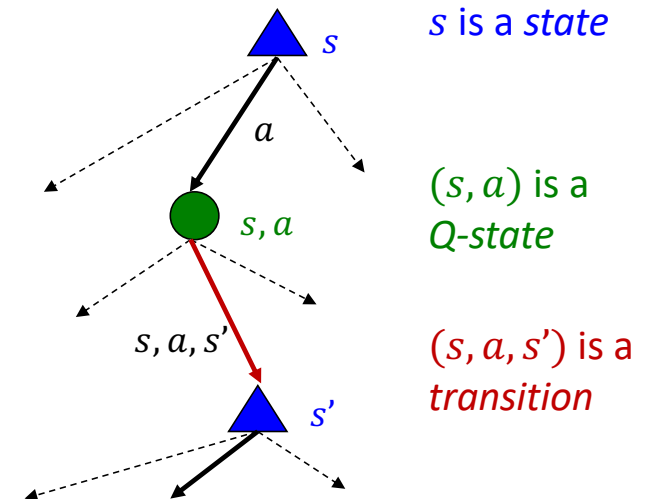
- Recursive definition of V and Q :

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



Example: Gridworld V^* Values



Noise = 0.2
Discount = 0.9
Living reward = 0

Computing Actions from Values

- Suppose we already have the optimal value function $V^*(s)$, how should the agent choose actions?

- One-step mini-expectimax:

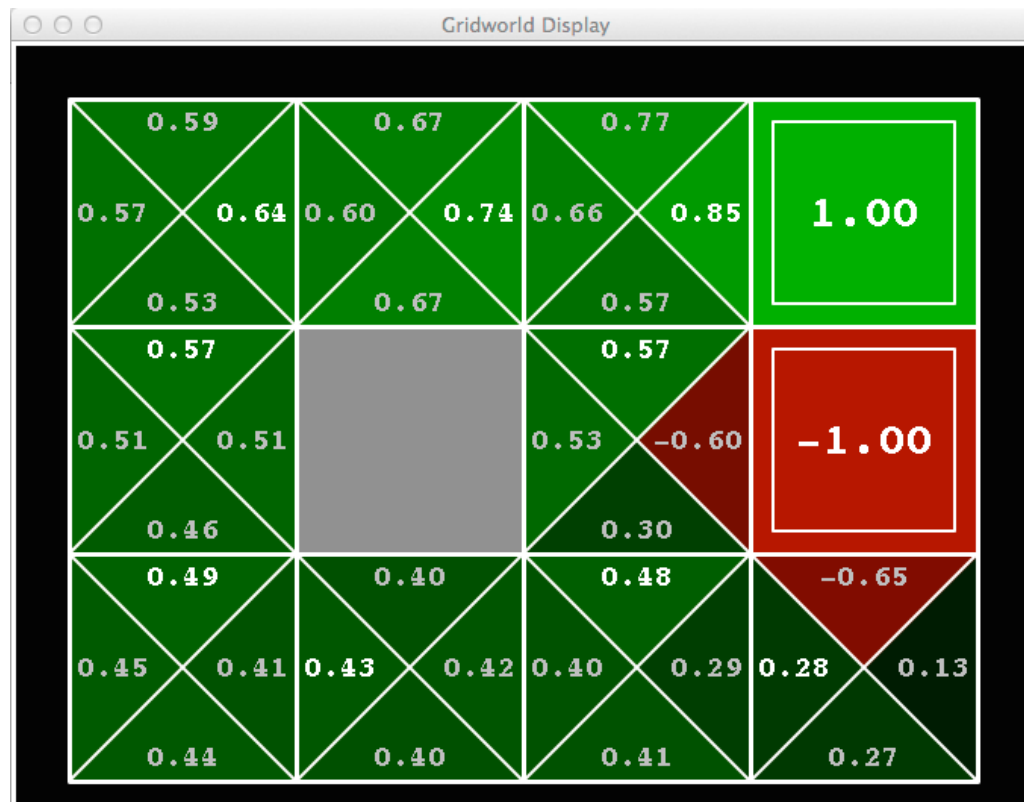
$$\pi^*(s) =$$

$$\arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- For each state s , evaluate all possible actions a ;
- Use the transition model and rewards to compute expected utility;
- Select the action that maximizes expected value.
- This is called **policy extraction**, since it gets the policy implied by the values.

0.95	0.96	0.98	1.00
0.94		0.89	-1.00
0.92	0.91	0.90	0.80

Example: Gridworld Q^* Values



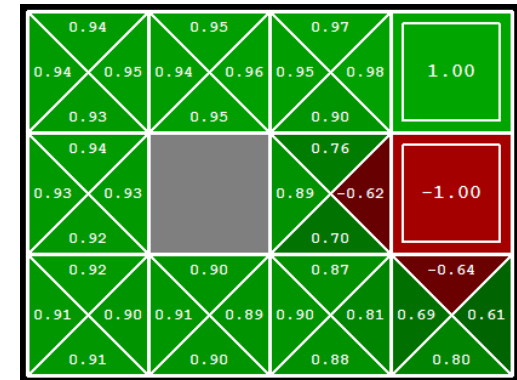
Noise = 0.2
Discount = 0.9
Living reward = 0

Computing Actions from Q-Values

- Let's imagine we have the optimal Q-values:
- How should we act?
 - Completely trivial to decide!

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

- Important lesson: actions are easier to select from Q-values than V-values!
- Then the question is how to calculate these values?
 - Value iteration, policy iteration.



Value Iteration

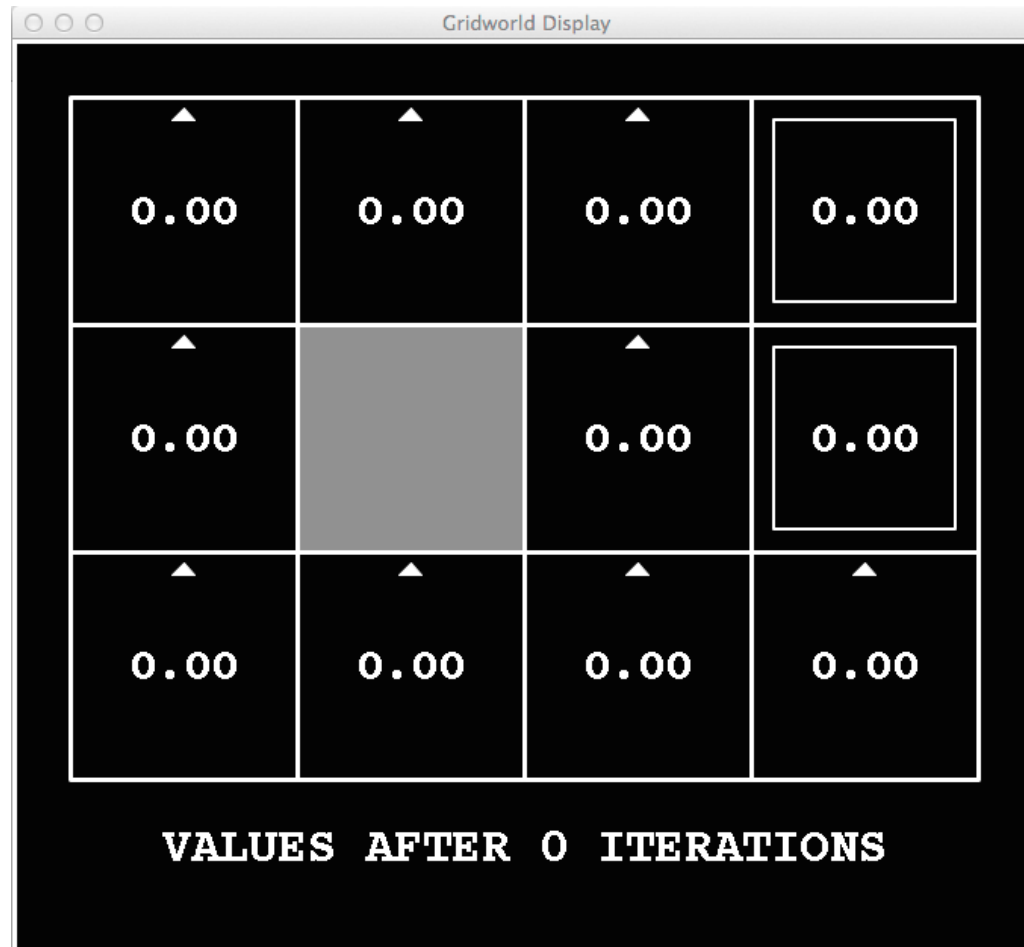
- Bellman equation **characterizes** the optimal values:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

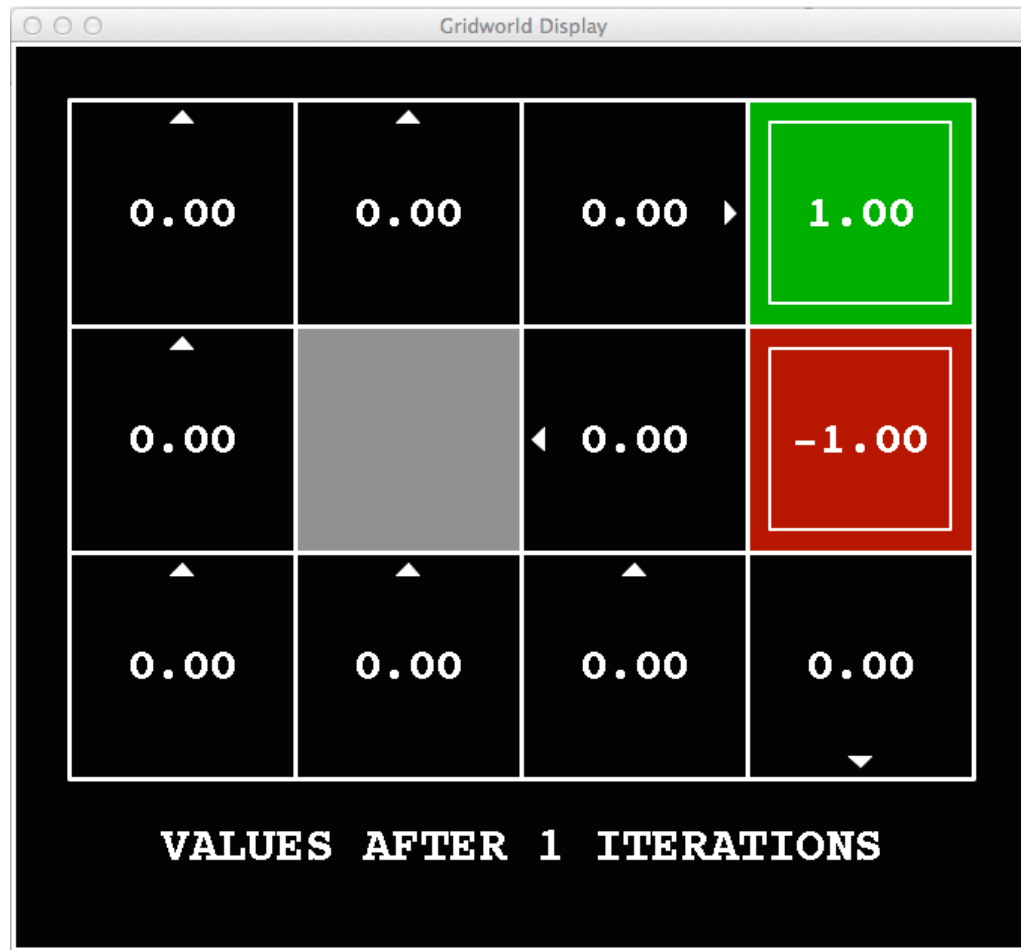
- **Value iteration:** compute the optimal value function $V^*(s)$ for each state:
 - Initialize all state values $V(s)$ to zero (or arbitrary values);
 - Iteratively update each state's value using the **Bellman update**;

$$V(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

- Repeat until values converge (i.e., changes are below a small threshold);
- Extract policy: for each state, choose the action that maximizes expected value.



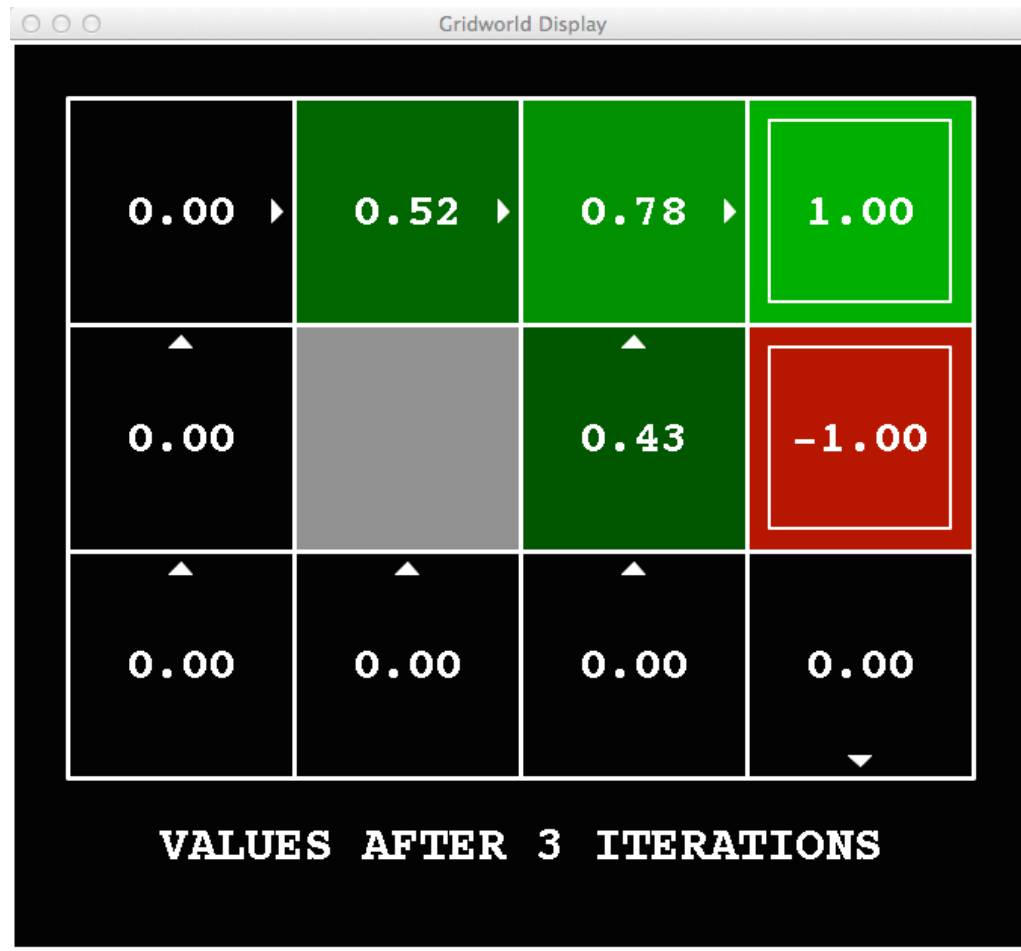
Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



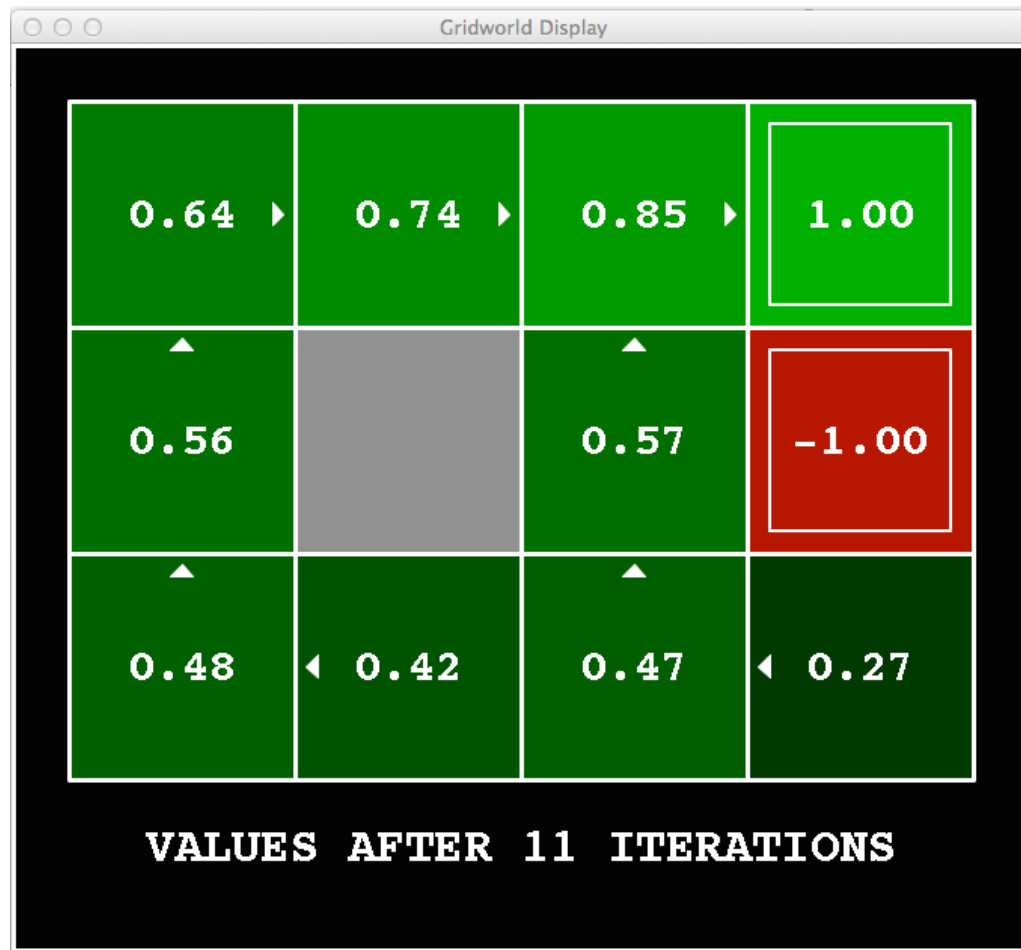
Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0

Problems with Value Iteration

- Value iteration repeats the Bellman updates:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Computational Cost:

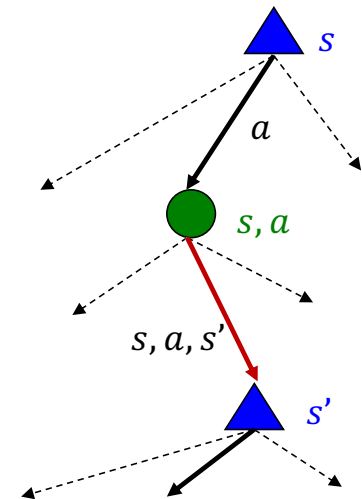
- Each iteration is slow: $O(S^2A)$ complexity;
- Especially expensive for large state spaces.

- Redundant Updates:

- The “max” action at each state often remains unchanged;
- Yet we recompute it every time.

- Delayed Efficiency:

- The policy may stabilize early;
- But value iteration continues updating values unnecessarily.



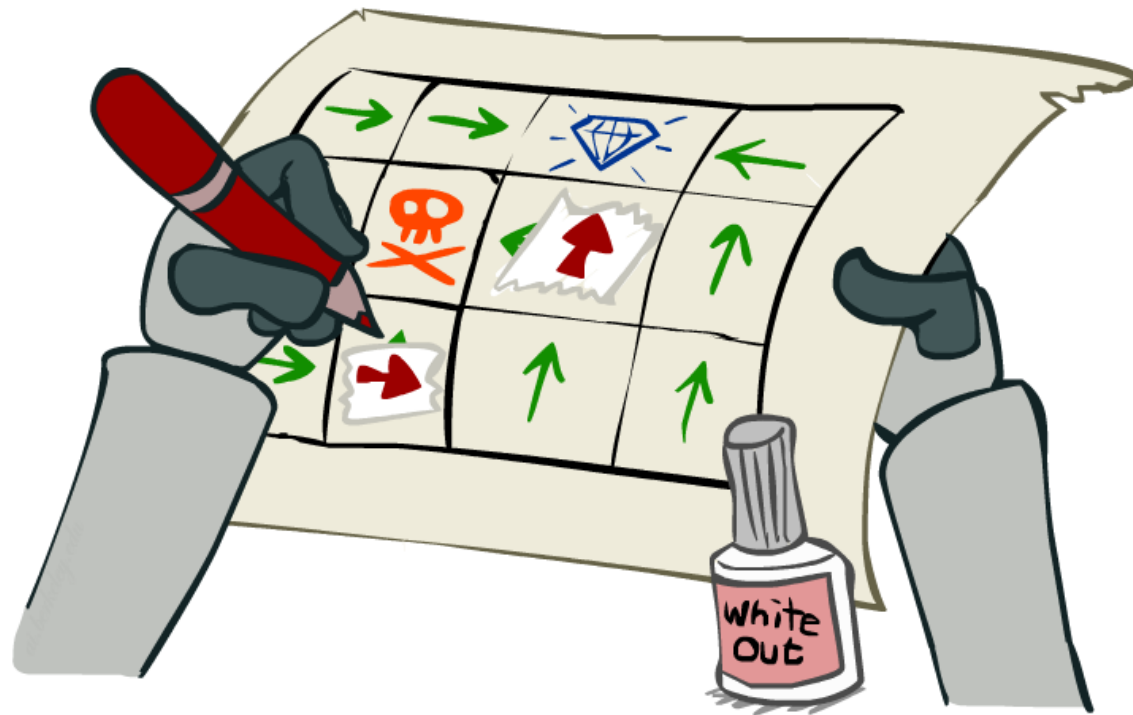


Noise = 0.2
Discount = 0.9
Living reward = 0



Noise = 0.2
Discount = 0.9
Living reward = 0

Policy Iteration

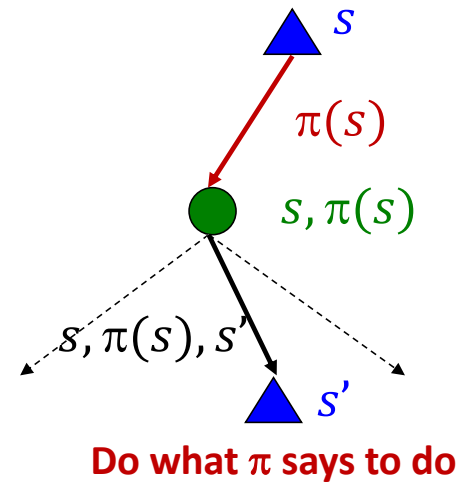
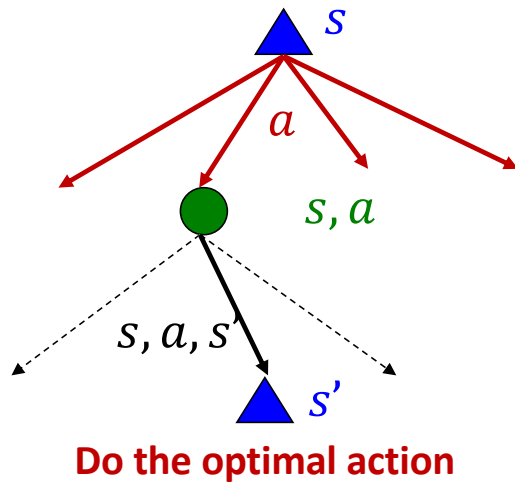


Policy Iteration

- Compute the optimal value function and derive the optimal policy:
 - Step 1: Policy **Evaluation**
 - Compute utilities for a fixed policy (not necessarily optimal);
 - Step 2: Policy **Improvement**
 - Use one-step lookahead to update the policy;
 - Apply improved actions based on the newly evaluated values;
 - **Repeat** until policy converges.
- Why Use It?
 - Still guarantees optimality;
 - Often converges faster than value iteration, especially when the policy stabilizes early.

Fixed Policy

- Expectimax trees evaluate all possible actions at each state to compute the optimal value function.
 - This involves taking the maximum over actions at every decision point
- If we fix a policy $\pi(s)$, the tree becomes simpler.
 - Only one action is considered per state, as dictated by the policy

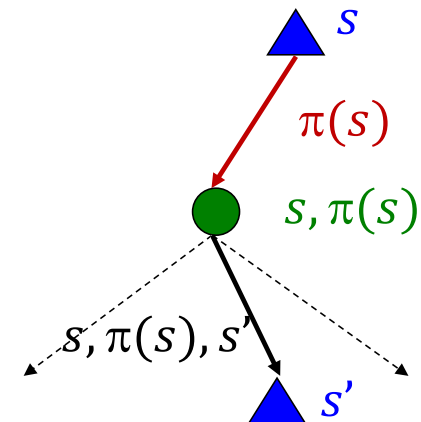


Policy Evaluation

- Compute the utility of a state s when following a fixed (not necessarily optimal) policy π :

- $V^\pi(s)$ = expected total discounted rewards starting in s and following π
- Recursive relation (one-step look-ahead / Bellman equation):

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]$$



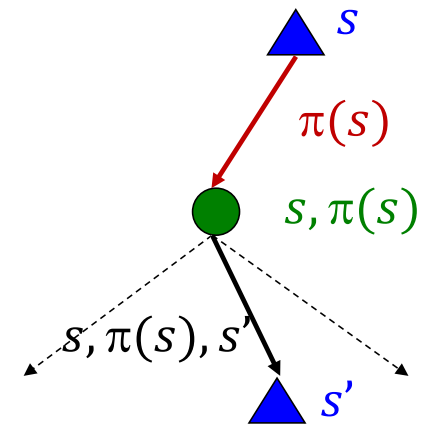
Policy Evaluation

- How do we calculate V for a fixed policy π ?
 - Idea 1: Turn recursive Bellman equations into updates (like value iteration)

$$V_0^\pi(s) = 0$$

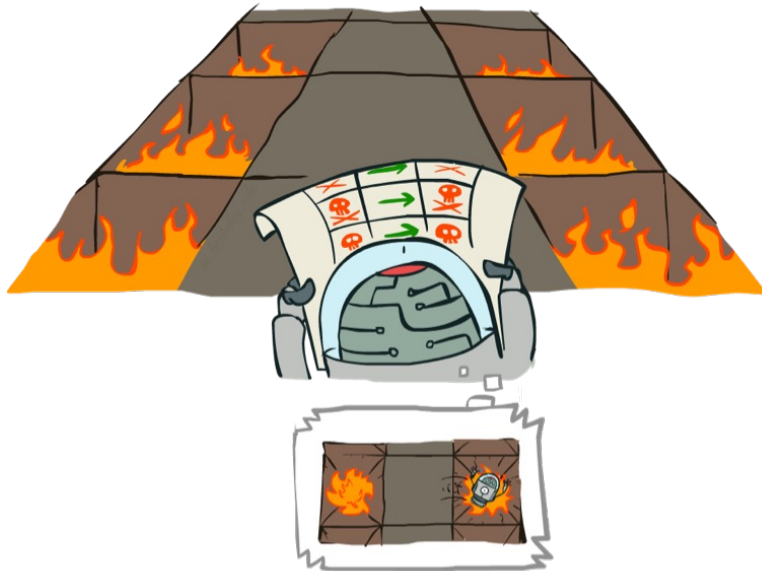
$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

- Idea 2: Without the max operation, the Bellman equations are just a linear system.
 - Solve the system of linear equations.

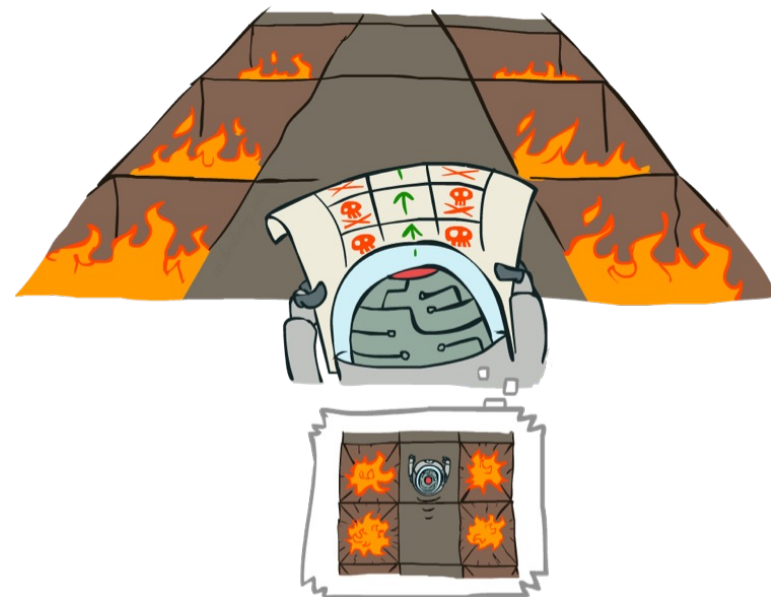


Example: Policy Evaluation

Always Go Right



Always Go Forward

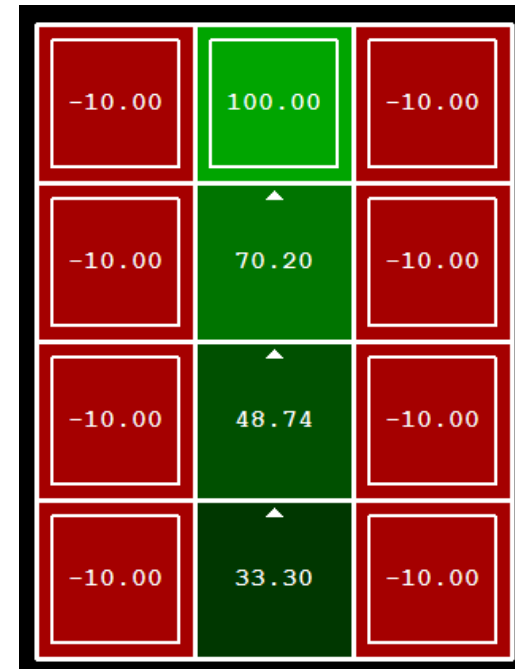


Example: Policy Evaluation

Always Go Right



Always Go Forward



Policy Improvement

- Evaluation (Step 1): For fixed current policy, find values with policy evaluation.
- Improvement (Step 2): For fixed values, get a better policy using policy extraction:
 - One-step look-ahead:

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$

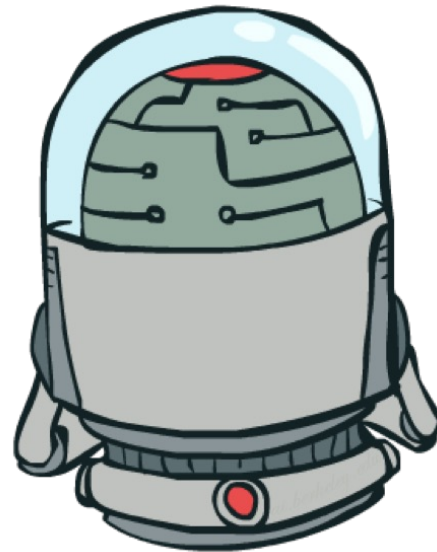
Value Iteration vs. Policy Iteration

- Both methods aim to compute the optimal value function and derive the optimal policy.
- In value iteration:
 - Every iteration updates both the values and (implicitly) the policy;
 - We don't track the policy, but taking the max over actions implicitly recomputes it.
- In policy iteration:
 - We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them);
 - After the policy is evaluated, a new policy is chosen (slow as value iteration);
 - The new policy will be better.

Summary: MDP Algorithms

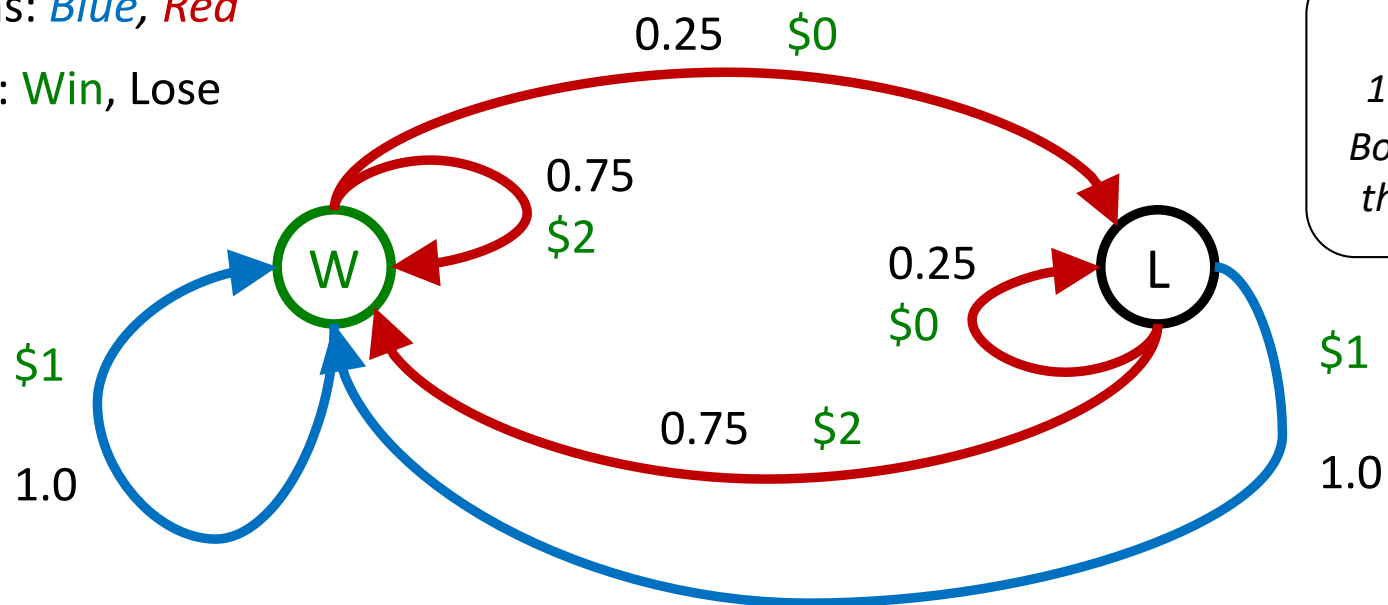
- So you want to....
 - Compute optimal values: use **value iteration** or **policy iteration**.
 - Compute values for a particular policy: use **policy evaluation**.
 - Turn your values into a policy: use **policy extraction** (one-step lookahead).
- These all look the same!
 - They basically are – they are all variations of Bellman updates.
 - They all use one-step lookahead expectimax fragments.
 - They differ only in whether we plug in a fixed policy or max over actions.

Double Bandits



Double-Bandit MDP

- Actions: *Blue*, *Red*
- States: *Win*, Lose



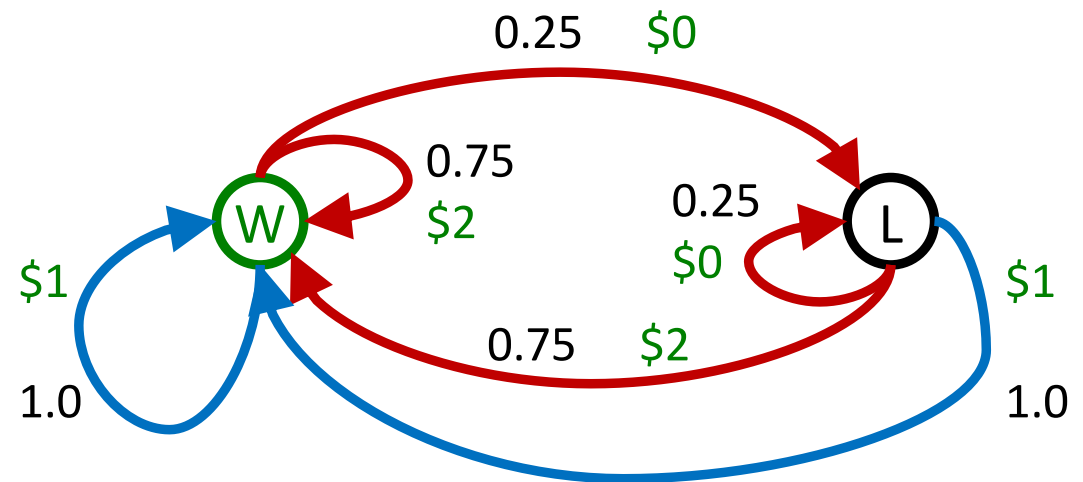
No discount
100 time steps
Both states have
the same value

Offline Planning

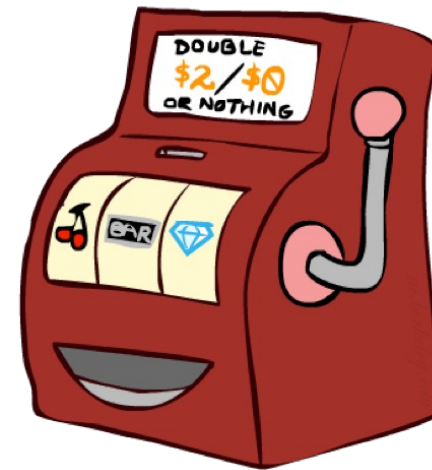
- Solving MDPs is offline planning.
 - You determine all quantities through computation.
 - You need to know the details of the MDP.
 - You do not actually play the game!

No discount
100 time steps
Both states have
the same value

	Value
Play Red	150
Play Blue	100



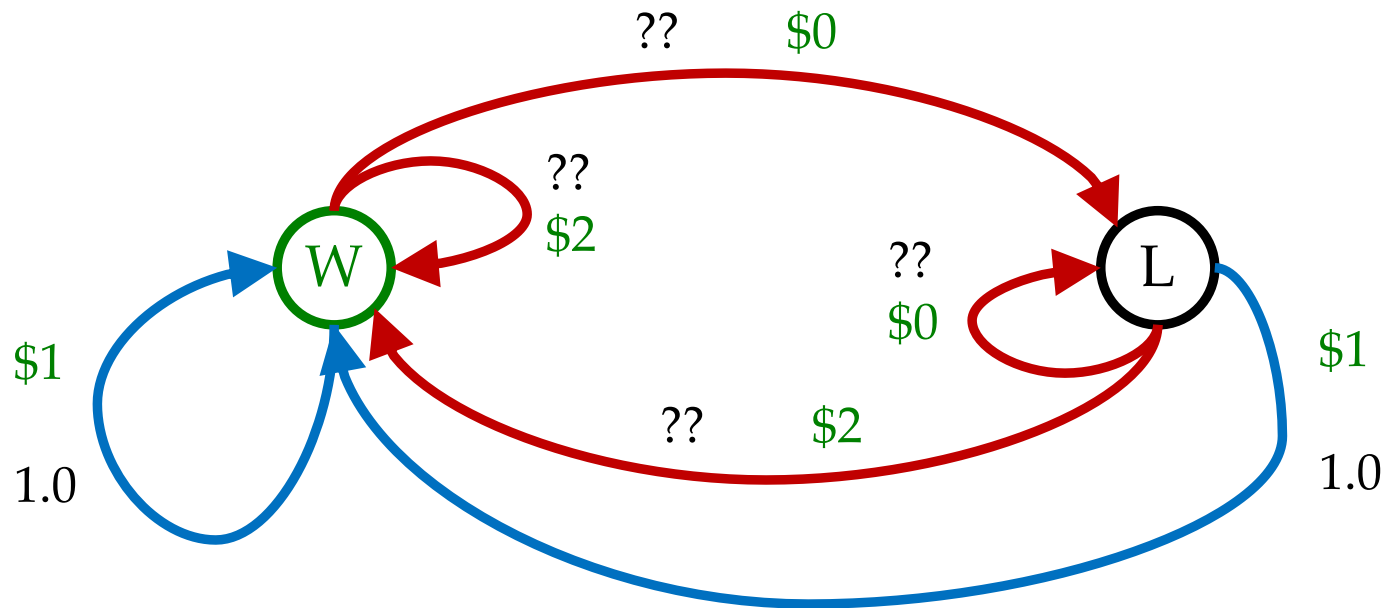
Let's Play!



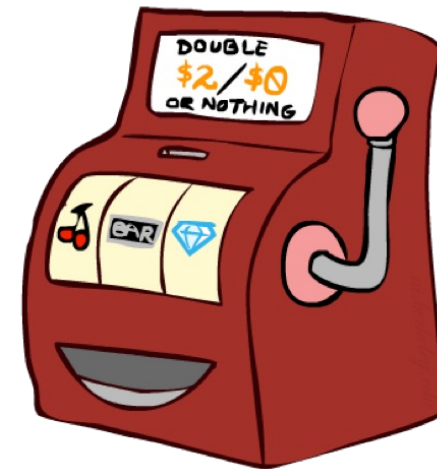
\$2 \$2 \$0 \$2 \$2
\$2 \$2 \$0 \$0 \$0

Online Planning

- Rules changed! Red's win chance is different.



Let's Play!



\$0 \$0 \$0 \$2 \$0

\$2 \$0 \$0 \$0 \$0

What Just Happened?

- That wasn't planning, it was learning!
 - Specifically, **reinforcement learning**.
 - There was an MDP, but you couldn't solve it with just computation.
 - You needed to actually act to figure it out.
- Important ideas in reinforcement learning that came up.
 - Exploration: you have to try unknown actions to get information.
 - Exploitation: eventually, you have to use what you know.
 - Regret: even if you learn intelligently, you make mistakes.
 - Sampling: because of chance, you have to try things repeatedly.
 - Difficulty: learning can be much harder than solving a known MDP.

Thank You!